Small Introduction To The Android SDK

Contents

1 Introduction **3**
1.1 Import the android SDK . . . 3

2 Implementation **4**
2.1 Getting started . . . 4
2.2 Authentication . . . 5
2.3 Open a book . . . 6
2.4 Headless Player . . . 7
2.4.1 SDK Player + Service . . . 7
2.4.2 A more customizable player . . . 7
2.5 Offline and Storage . . . 8
2.5.1 Offline Model . . . 8
2.5.2 Download Lifecycle . . . 8
2.5.3 Removing Downloaded Material . . . 9

3 Additional Info **9**

1 Introduction

A brief introduction to the WeDoBooks SDK, including setup and usage. Presented as a step-by-step guide in Android Studio[1].

1.1 Import the android SDK

To set up the SDK, first add the WeDoBooks Maven repository entry to the list of repositories. In newer projects this is typically configured in `settings.gradle` (or `settings.gradle.kts`). In older projects it may be placed in the project-level `build.gradle`, and it can also be added in the module-level `build.gradle` if required. See Listing 1.

```
    repositories {
        ... // Other repositories
        maven {
            url = uri("https://wdb-android-maven-844218222632.europe-west3.run.app")
            credentials {
                username = getLocalProperty("WDB_USER_NAME") // Provided by wedobooks
                password = getLocalProperty("WDB_PASSWORD") // Provided by wedobooks
            }
        }
}
```

Listing 1: settings.gradle snippet

Next, add the SDK to the modules' dependency lists, which can be found in `build.gradle`. See Listing 2.

```
    dependencies {
        implement "io.wedobooks:sdk:{version_number}"
    }
```

Listing 2: build.gradle (module) snippet

[1] https://developer.android.com/studio

2 Implementation

2.1 Getting started

This section describes how to implement the SDK. Before using the SDK, call the setup function to initialize it; this enables SDK functionality. See Listing 3.

```
WeDoBooksSDK.setup(
    context = this.applicationContext,
    config = WeDoBooksConfiguration(
        applicationId = BuildConfig.APPLICATION_ID,
        firebaseApiKey = {find in firebase project}
        firebaseProjectId = {find in firebase project},
        firebaseAppId = {find in firebase project},
        readerApiKey = BuildConfig.READER_API_KEY, \\ Provided by WeDoBooks
        readerApiSecret = BuildConfig.READER_API_SECRET \\ Provided by WeDoBooks
    ),
    themeConfig = WeDoBooksThemeConfiguration
        .builder()
        .build()
)
```

Listing 3: WDBSdk setup

The `WeDoBooksThemeConfiguration` in Listing 3 can be used to configure theme colors and fonts applied by the reader/player.

2.2 Authentication

Most SDK features require an authenticated user. Do note that user authentication requires a backend-to-backend integration.
After the integration is in place, call `WeDoBooks.userOperations.signInWithToken`. See Figure 1 for an overview of the flow.

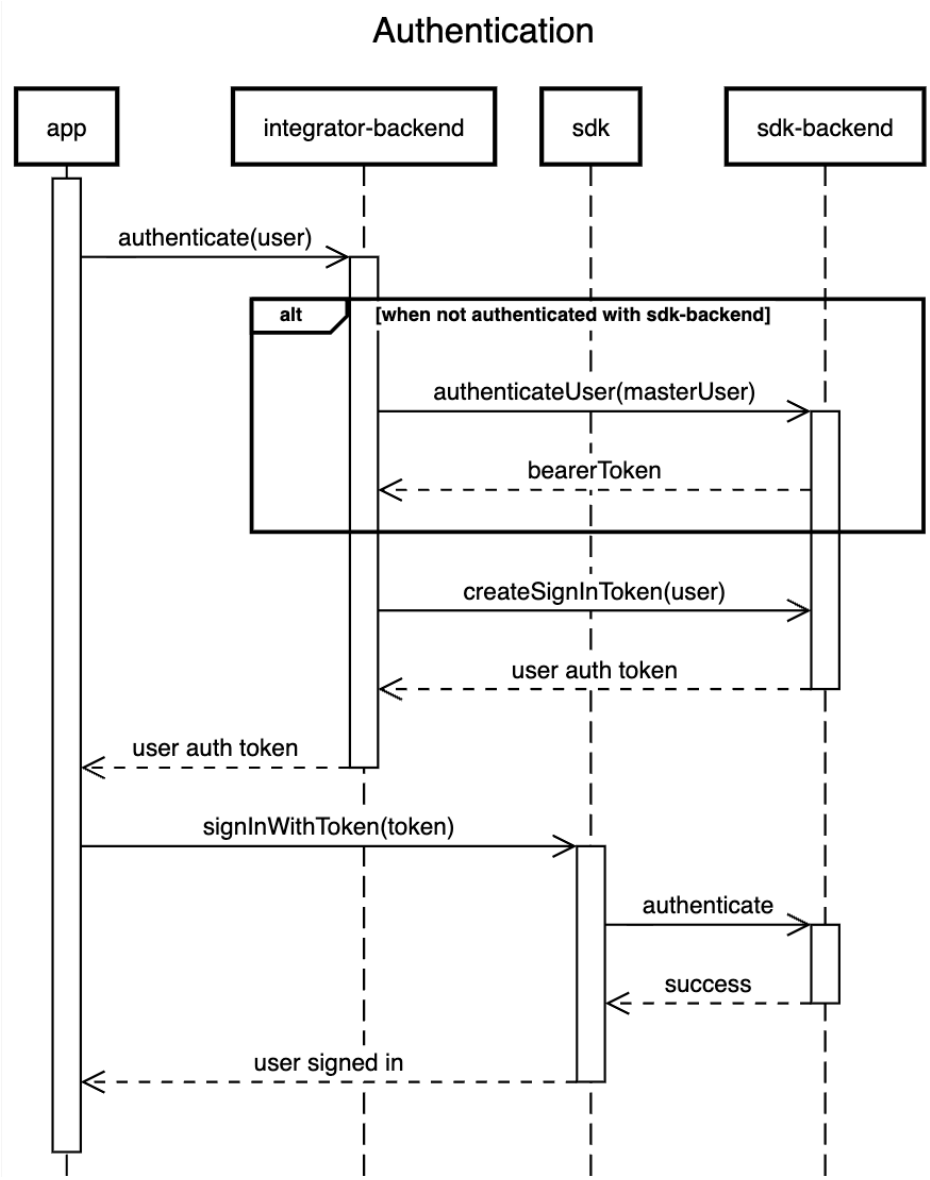

Figure 1: Authentication sequence diagram

2.3 Open a book

Provide an ISBN and call `WeDoBooksSDK.bookOperations.checkoutBook(isbn)`. The call either creates a new checkout or returns an existing one if the user already has a checkout for the given ISBN.
After obtaining the checkout, the reader can be opened. The SDK automatically launches either the audio player or the e-book reader based on the ISBN. Listing 4 shows how to obtain the reader view, and Figure 2 provides an overview of the flow.

```
// you can set your own coverUrl else set to null if you want to use coverUrl
    provided by WeDoBooks
WeDoBooksSDK.bookOperations.BookScreen(
    checkout = checkout,
    cover = null,
    onCloseClick = {}, // implement your own close button action
    isFinishButtonEnabled = false, // there is a button when you get to the end of
        the ebook
    onFinishClick = {}, // action for finish button
    onAudioMinimizeClick = null, // alternative close button for audioplayer
    viewModelStoreOwner = null, // if you want to save state outside this
        composable
    isDarkMode = isDarkMode
)
```

Listing 4: BookScreen

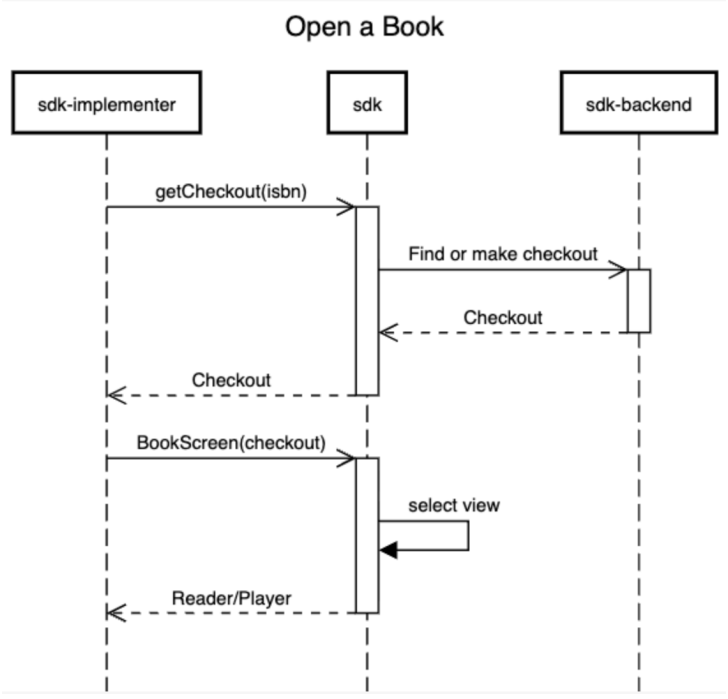

Figure 2: Open a book

2.4 Headless Player

For audiobooks, it is possible to use a custom UI instead of the one provided by the SDK. There are two ways to achieve this:

2.4.1 SDK Player + Service

The SDK provides a built-in player, which is the same implementation used in the BookScreen (see Listing 4). To integrate with this player, use `WeDoBooksSDK.headlessAudioPlayer.playerUiState`, which exposes a Flow that can be observed for UI updates. To start playback, call `WeDoBooksSDK.headlessAudioPlayer.loadAudioBook(checkout)`. This method loads and prepares the audiobook (see Listing 5).

```
// first Load the book, this throws in case it cannot load
WeDoBooksSDK.headlessAudioPlayer.LoadAudioBook(
    checkout = checkout,
    cover = null,
    initialProgressMs = null
)
```

Listing 5: Loading and preparing an audio book

Observe `playerUiState` to determine when the player is ready. Once ready, use `WeDoBooksSDK.headlessAudioPlayer.withAudioController` to control playback (see Listing 6).

```
// withAudioController is how you access the controller
WeDoBooksSDK.headlessAudioPlayer.withAudioController {
    it.play()
}
```

Listing 6: play/pause with audio controller

2.4.2 A more customizable player

The SDK also offers a more customizable player implementation. This player is based on the Media3 Player interface. Note that this implementation does not include a foreground service by default. As a result, it is not configured for background playback or Android Auto support out of the box. An example setup for these scenarios is available in the sample application[2]. To get started with this player, refer to Listing 7.

[2] https://github.com/wedobooks/wedobooks-sdk-android-sample

```
// use the builder to start
// the WdbAudioPlayer interfaces the media3 Player interface
val player: WdbAudioPlayer = WeDoBooksSdk.wdbAudioPlayerBuilder().build()
player.loadBook(checkout)
```

Listing 7: media3 interface Player builder

2.5 Offline and Storage

2.5.1 Offline Model

Downloaded material is scoped to the current signed-in user. The SDK stores downloaded ebook and audio artifacts in app storage, and stores ebook offline keys in the datastore. Ebooks are downloaded before they are opened, so they are available offline from that point until the material is explicitly removed through the API. Audiobooks are downloaded through a call to the API. Once a download completes successfully, the book can be opened while offline.

2.5.2 Download Lifecycle

A download begins by acquiring a checkout, which can be obtained through `WeDoBooksSDK.bookOperations.checkoutBook(isbn)` or `WeDoBooksSDK.bookOperations.allCheckoutsFlow`. The checkout is then passed to `WeDoBooksSDK.storageOperations.downloadBook(checkout)`; refer to listing 8. Downloaded material is opened the same way as a book that has not been downloaded.

```
suspend fun downloadBook() {
    val checkout = WeDoBooksSdk.bookOperations.checkoutBook(isbn)
    try {
        WeDoBooksSdk.storageOperations.downloadBook(checkout)
    } catch (e: Throwable) {
        Log.d("error", e?.message.orEmpty())
    }
}
```

Listing 8: Download book

Download progress can be tracked through `WeDoBooksSDK.storageOperations.bookDownloadsFlow`. See listing 9

```
val isbn = checkout.materialId
WeDoBooksSdk.storageOperations.bookDownloadsFlow
        .map { it[isbn] }
        .filter { it != null }
        .onEach {
            Log.d("progress", " ${it?.progress ?: 0}")
        }
        .launchIn(viewModelScope)
```

Listing 9: Download progress

The flow also reports the current status of the book, including whether the download failed.

2.5.3 Removing Downloaded Material

Books are removed using `WeDoBooksSDK.storageOperations.removeBook(isbn)` or `WeDoBooksSDK.storageOperations.removAll()`. Upon successful removal, the key for that book is no longer present in `bookDownloadsFlow`.

3 Additional Info

The SDK provides more than obtaining book checkouts and launching the reader. For concrete examples, refer to the sample application[3], which demonstrates user reading statistics and the implementation of a "currently playing" bar.
For additional customization options and integration points, consult the SDK API documentation[4]. The API exposes extra endpoints and configuration for checkouts and reading statistics.

[3] https://github.com/wedobooks/wedobooks-sdk-android-sample
[4] https://sdk-stage-bokus-api-gateway-7x7f3u67.nw.gateway.dev/docs